암호화 시스템 및 키 관리

개요

IPFS는 분산 네트워크에서 안전한 통신과 데이터 무결성을 보장하기 위해 강력한 암호화 시스템을 사용합니다. 이 문서는 IPFS의 암호화 구조, 키 관리 방식, 그리고 보안 메커니즘을 상세히 설명합니다.

암호화 알고리즘 지원

지원되는 키 타입

IPFS는 다양한 암호화 알고리즘을 지원하여 보안 요구사항과 성능 특성에 따라 선택할 수 있습니다:

1. RSA (Rivest-Shamir-Adleman)

```
// config/init.go에서 RSA 키 생성
case "rsa":
if settings.Size == -1 {
settings.Size = options.DefaultRSALen // 기본 2048비트
}
fmt.Fprintf(out, "generating %d-bit RSA keypair...", settings.Size)
priv, pub, err := crypto.GenerateKeyPair(crypto.RSA, settings.Size)
```

특징:

- **키 크기**: 1024, 2048, 4096비트 지원 (기본값: 2048비트)

- **용도**: 전통적인 공개키 암호화, 디지털 서명
- **성능**: 상대적으로 느리지만 널리 호환됨
- **보안**: 충분한 키 크기 사용 시 안전

2. Ed25519 (Edwards-curve Digital Signature Algorithm)

```
// Ed25519 키 생성
case "ed25519":
if settings.Size != -1 {
return ident, fmt.Errorf("number of key bits does not apply when using ed25519
}
fmt.Fprintf(out, "generating ED25519 keypair...")
priv, pub, err := crypto.GenerateEd25519Key(rand.Reader)
```

특징:

- **키 크기**: 고정 256비트 (32바이트)
- **용도**: 고성능 디지털 서명
- **성능**: RSA 대비 매우 빠른 서명/검증
- **보안**: 타원곡선 기반의 강력한 보안

3. Secp256k1 (비트코인 호환)

특징:

- **키 크기**: 256비트
- **용도**: 비트코인 생태계와의 호환성
- **성능**: Ed25519와 유사한 성능

- **보안**: ECDSA 기반 보안

알고리즘 선택 기준

| 알고리즘 | 키 크기 | 서명 속도 | 검증 속도 | 호환성 | 권장 용도 |

|---|---|---|---|---|---|

| RSA-2048 | 2048비트 | 느림 | 빠름 | 높음 | 레거시 호환성 |

| RSA-4096 | 4096비트 | 매우 느림 | 보통 | 높음 | 고보안 요구사항 |

| Ed25519 | 256비트 | 매우 빠름 | 매우 빠름 | 보통 | 일반적 사용 (권장) |

| Secp256k1 | 256비트 | 빠름 | 빠름 | 블록체인 | 암호화폐 통합 |

키 생성 및 관리

키 생성 과정

IPFS 노드 초기화 시 키 생성 과정:

```
// CreateIdentity 함수의 키 생성 과정
func CreateIdentity(out io.Writer, opts []options.KeyGenerateOption) (Identity,
settings, err := options.KeyGenerateOptions(opts...)
if err != nil {
return ident, err
}

var sk crypto.PrivKey
var pk crypto.PubKey
```

```go
// 알고리즘별 키 생성
switch settings.Algorithm {
case "rsa":
priv, pub, err := crypto.GenerateKeyPair(crypto.RSA, settings.Size)
sk = priv
pk = pub
case "ed25519":
priv, pub, err := crypto.GenerateEd25519Key(rand.Reader)
sk = priv
pk = pub
}

// 개인키 직렬화 및 Base64 인코딩
skbytes, err := crypto.MarshalPrivateKey(sk)
if err != nil {
return ident, err
}
ident.PrivKey = base64.StdEncoding.EncodeToString(skbytes)

// Peer ID 생성
id, err := peer.IDFromPublicKey(pk)
if err != nil {
```

```
return ident, err
}
ident.PeerID = id.String()
return ident, nil
}
```

키 저장 구조

1. 설정 파일 구조

```
// config/identity.go
type Identity struct {
PeerID string // Base58 인코딩된 Peer ID
PrivKey string // Base64 인코딩된 개인키 (현재 암호화되지 않음)
}
```

2. 키 저장 위치

- **설정 파일**: `~/.ipfs/config` (JSON 형식)
- **권한**: 파일 시스템 권한으로 보호 (600)
- **암호화**: 현재 평문 저장 (TODO: 암호화 구현 예정)

```
{
"Identity": {
"PeerID": "12D3KooWGRPLCW...",
"PrivKey": "CAESQNlX4ekFoTnXIFEjDQtzuBQ7AOzk4uHBvTKynjE..."
```

```
}

}
```

키 보안 메커니즘

1. 현재 보안 상태

```
// config/identity.go의 TODO 주석

// currently storing key unencrypted. in the future we need to encrypt it.

// TODO(security)
```

현재 구현:

- 개인키가 Base64로 인코딩되어 평문 저장
- 파일 시스템 권한으로만 보호
- 메모리에서 평문으로 처리

2. 계획된 보안 강화

- **키 암호화**: 패스프레이즈 기반 키 암호화
- **하드웨어 보안 모듈**: HSM 통합 지원
- **키 분할**: Shamir's Secret Sharing 지원

키 검증 및 무결성

1. 키 쌍 일치성 검증

```
// core/node/identity.go
```

```
func PrivateKey(sk crypto.PrivKey) func(id peer.ID) (crypto.PrivKey, error) {
return func(id peer.ID) (crypto.PrivKey, error) {
id2, err := peer.IDFromPrivateKey(sk)
if err != nil {
return nil, err
}

if id2 != id {
return nil, fmt.Errorf("private key in config does not match id: %s != %s", id,
}
return sk, nil
}
}
```

2. 키 복구 및 디코딩

```
// DecodePrivateKey 함수
func (i *Identity) DecodePrivateKey(passphrase string) (ic.PrivKey, error) {
pkb, err := base64.StdEncoding.DecodeString(i.PrivKey)
if err != nil {
return nil, err
}

// 현재는 암호화되지 않은 키 처리
```

```
return ic.UnmarshalPrivateKey(pkb)
}
```

키 회전 및 생명주기 관리

키 회전 전략

1. 정기적 키 회전

- **권장 주기**: RSA 키는 2-3년, Ed25519 키는 5년
- **자동화**: 현재 수동 프로세스, 자동화 계획 중
- **호환성**: 기존 Peer ID 유지를 위한 점진적 전환

2. 키 마이그레이션

```
# 새로운 키로 노드 재초기화
ipfs init --algorithm ed25519

# 기존 데이터 마이그레이션
ipfs repo migrate
```

키 백업 및 복구

1. 키 백업 방법

```
# 설정 파일 백업
```

```
cp ~/.ipfs/config ~/.ipfs/config.backup


# 개인키만 추출
ipfs config Identity.PrivKey
```

2. 키 복구 절차

```
# 설정 파일 복원
cp ~/.ipfs/config.backup ~/.ipfs/config


# 키 검증
ipfs id
```

암호화 성능 최적화

성능 벤치마크

| 작업 | RSA-2048 | Ed25519 | 성능 비율 |

|------|----------|---------|----------|

| 키 생성 | 100ms | 1ms | 100:1 |

| 서명 생성 | 5ms | 0.05ms | 100:1 |

| 서명 검증 | 0.5ms | 0.1ms | 5:1 |

| 키 크기 | 2048비트 | 256비트 | 8:1 |

최적화 권장사항

1. 알고리즘 선택

- **신규 노드**: Ed25519 사용 권장
- **기존 노드**: 점진적 RSA → Ed25519 마이그레이션
- **특수 요구사항**: Secp256k1 (블록체인 통합 시)

2. 성능 튜닝

```
// 키 캐싱으로 성능 향상
var keyCache = make(map[string]crypto.PrivKey)

func getCachedKey(peerID string) crypto.PrivKey {
if key, exists := keyCache[peerID]; exists {
return key
}
// 키 로드 및 캐싱
return loadAndCacheKey(peerID)
}
```

보안 모범 사례

1. 키 관리 보안

- **파일 권한**: 설정 파일을 600 권한으로 설정
- **메모리 보안**: 사용 후 키 데이터 메모리에서 제거
- **로깅 제한**: 개인키 정보 로그 출력 금지

2. 네트워크 보안

- **TLS 사용**: libp2p의 TLS 1.3 지원 활용
- **키 교환**: ECDH 기반 안전한 키 교환
- **전방향 보안**: 세션 키 정기 갱신

3. 운영 보안

- **정기 감사**: 키 사용 패턴 모니터링
- **침해 대응**: 키 손상 시 즉시 회전
- **백업 암호화**: 키 백업 파일 암호화 저장

참고 자료

- 신원 관리 시스템
- 콘텐츠 차단 메커니즘
- libp2p 암호화 문서